Snowflake CLI

Snowflake CLI is next gen command line utility to interact with Snowflake.

> [!NOTE]
>
> - All commands has `--help` option
> - All command allows output format to be `table`(**default**) or `json`.

Install

```
pip install -U snowflake-cli-labs
```

or

```
pipx install snowflake-cli-labs
```

Get Help

```
snow --help
```

Information

General information about version of CLI and Python, default configuration path etc.,

```
snow  --info
```

> [!TIP] The connection configuration `config.toml` by default is stored under`$HOME/.snowflake`. If you wish to change it set the environment variable `$SNOWFLAKE_HOME`[1] to director where you want to store the `config.toml`

Connection

Add

```
snow connection add
```

Adding connection `cheatsheets` following the prompts,

```
snow connection add
```

Adding connection using command options,

```
snow connection add --connection-name cheatsheets \
  --account <your-account-identifier> \
  --user <your-user> \
  --password <your-password>
```

> [!NOTE] Currently need to follow the prompts for the defaults or add other parameters

List

```
snow connection list
```

Set Default

```
snow connection set-default cheatsheets
```

Test a Connection

```
snow connection test -c cheatsheets
```

> [!TIP] If you don’t specify `-c`, then it test with default connection that was set in the config

Cortex

Supported LLMs[2]

- Large
  - reka-core
  - llama3-70b
  - mistral-large
- Medium
  - snowflake-arctic(**default**)
  - reka-flash

    - mixtral-8x7b
    - llama2-70b-chat
- Small
    - llama3-8b
    - mistral-7b
    - gemma-7b

Complete

Generate a response for a given prompt,

```
snow cortex complete "Tell me about Snowflake"
```

With a specific supported LLM,

```
snow cortex complete "Tell me about Snowflake" --model=mistral-7b
```

With history,

```
snow cortex complete --file samples/datacloud.json
```

Extract Answer

Get answer for the question from a text,

```
snow cortex extract-answer 'what does snowpark do ?' 'Snowpark
provides a set of libraries and runtimes in Snowflake to securely
deploy and process non-SQL code, including Python, Java and
Scala.'
```

Get answers for the questions from a text file,

```
snow cortex extract-answer 'What does Snowflake eliminate?'  --
file samples/answers.txt

snow cortex extract-answer 'What non-SQL code Snowpark process?'
--file samples/answers.txt
```

Sentiment

Sentiment Score	Sentiment
1	Positive
-1	Negative

A positive sentiment (score: `0.64`) from a text,

```
snow cortex sentiment 'Snowflake is a awesome company to work.'
```

A negative sentiment (approx score `-0.4`) from a text,

```
snow cortex sentiment --file samples/sentiment.txt
```

Summarize

From a text,

```
snow cortex summarize 'SnowCLI is next gen command line utility to
interact with Snowflake. It supports manipulating lot of Snowflake
objects from command line.'
```

From a file,

```
snow cortex summarize --file samples/asl_v2.txt
```

Translate

Currently supported languages

- English(`en`)
- French(`fr`)
- German(`de`)
- Polish(`pl`)
- Japanese(`ja`)
- Korean(`ko`)
- Italian(`it`)
- Portuguese(`pt`)
- Spanish(`es`)
- Swedish(`sv`)
- Russian(`ru`)

Translate from English to French a text,

```
snow cortex translate --from en --to fr 'snowflake is an awesome
company to work for.'
```

Translate from English to Spanish a text from a file,

```
snow cortex translate --from en --to es --file samples/
translate.txt
```

Work with Snowflake Objects using SQL

Creating Objects

Simple one line query,

```
snow sql -q 'CREATE DATABASE FOO'
```

Loading DDL/DML commands from a file,

```
snow sql --filename my_objects.sql
```

Using Standard Input(`stdin`)

```
cat <<EOF | snow sql --stdin
CREATE OR REPLACE DATABASE FOO;
USE DATABASE FOO;
CREATE OR REPLACE SCHEMA CLI;
USE SCHEMA CLI;
CREATE OR ALTER TABLE employees(
  id int,
  first_name string,
  last_name string,
  dept int
);
EOF
```

Listing Objects

Use the following command to see the list of supported objects,

```
snow object list --help
```

Warehouses

List all available warehouses for the current role,

```
snow object list warehouse
```

Databases

List all available databases for the current role,

```
snow object list database
```

List all databases in `JSON` format,

```
snow object list database --format json
```

> [!TIP] With `JSON` you can extract values using tools like jq
> e.g. to get only names of the databases
>
> ```
> snow object list database --format json | jq '.[].name'
> ```

List databases that starts with `snow`,

```
snow object list database --like '%snow%'
```

Schemas

List all schemas,

```
snow object list schema
```

Filtering schemas by database named `foo`,

```
snow object list schema --in database foo
```

Tables

List all tables

```
snow object list table
```

List tables in a specific schema `cli` of a database `foo`,

```
snow object list table --database foo --in schema cli
```

Describe an Object

Let us describe the table `employees` in the `foo` database' schema `cli`,

```
snow object describe table employees --database foo --schema cli
```

Drop an object

Drop an table named `employees` in schema `cli` of database `foo`,

```
snow object drop table employees --database foo --schema cli
```

Streamlit Applications

Create a Streamlit application and deploy to Snowflake,

```
snow streamlit init streamlit_app
```

Create a warehouse that the Streamlit application will use,

```
snow sql -q 'CREATE WAREHOUSE my_streamlit_warehouse'
```

Create a database that the Streamlit application will use,

```
snow sql -q 'CREATE DATABASE my_streamlit_app'
```

Deploy an Application

> [!IMPORTANT] Ensure you are in the Streamlit application folder before running the command.

```
snow streamlit deploy --database=my_streamlit_app
```

List Applications

List all available streamlit applications,

```
snow streamlit list
```

Describe Application

Get details about a streamlit application `streamlit_app` in schema of `public` of database`my_streamlit_app`,

```
snow streamlit describe streamlit_app --schema=public --
database=my_streamlit_app
```

> [!NOTE] When describing Streamlit application either provide the schema as parameter or use fully qualified name

Get Application URL

Get the streamlit application URL i.e. the URL used to access the hosted application,

```
snow streamlit get-url streamlit_app --database=my_streamlit_app
```

Drop Application

Drop a streamlit application named `streamlit_app` in schema of `public` of database`my_streamlit_app`,

```
snow streamlit drop streamlit_app --schema=public --
database=my_streamlit_app
```

Stages

SnowCLI allows managing the internal stages.

Create

Create a stage named `cli_stage` in schema `cli` of database `foo`,

```
snow stage create cli_stage  --schema=cli --database=foo
```

Describe

Get details of stage,

```
snow stage describe cli_stage  --schema=cli --database=foo
```

List Stages

List all available stages,

```
snow stage list
```

List stages in specific to a database named `foo`,

```
snow stage list --in database foo
```

List stages by name that starts with `cli` in database `foo`,

```
snow stage list --like 'cli%' --in database foo
```

Copy Files

Download employees.csv,

```
curl -sSL -o employees.csv https://raw.githubusercontent.com/
Snowflake-Labs/sf-cheatsheets/main/samples/employees.csv
```

Copy `employees.csv` to stage `cli_stage` to a path `/data`,

```
snow stage copy employees.csv '@cli_stage/data'  --schema=cli --
database=foo
```

List Files in Stage

List all files in stage `cli_stage` in schema `cli` of database `foo`,

```
snow stage list-files cli_stage  --schema=cli --database=foo
```

List files by pattern,

```
snow stage list-files cli_stage --pattern='.*[.]csv' --schema=cli
--database=foo
```

Execute Files From Stage

Download the load_employees.sql,

```
curl -sSL -o load_employees.sql https://raw.githubusercontent.com/
Snowflake-Labs/sf-cheatsheets/main/samples/load_employees.sql
```

Copy `load_employees.sql` to stage `cli_stage` at path `/sql`,

```
snow stage copy load_employees.sql '@cli_stage/sql'  --schema=cli
--database=foo
```

Execute the SQL[3] from stage,

```
snow stage execute '@cli_stage/sql/load_employees.sql'  --
schema=cli --database=foo
```

> [!NOTE] Execute takes the glob pattern, allowing to specify the file pattern to execute. `@stage/*` or `@stage/*.sql` both executes only sql files

Query all employees to make sure the load worked,

```
snow sql --schema=cli --database=foo -q 'SELECT * FROM EMPLOYEES'
```

Download variables.sql,

```
curl -sSL -o variables.sql https://raw.githubusercontent.com/
Snowflake-Labs/sf-cheatsheets/main/samples/variables.sql
```

Copy the `variables.sql` to stage,

```
snow stage copy  variables.sql '@cli_stage/sql' --schema=cli --
database=foo
```

Execute files from stage with values for template variables(`{{.dept}}` in variables.sql),

```
snow stage execute '@cli_stage/sql/variables.sql' --
variable="dept=1"  --schema=cli --database=foo
```

Executing `variables.sql` would have created a view named `EMPLOYEE_DEPT_VIEW`, list the view it to see the variables replaced,

```
snow object list view --like 'emp%' --database=foo --schema=cli
```

> [!NOTE] SnowCLI allows processing templating using `{{...}}` and `&{...}`
>
> - `{{...}}` is a preferred templating i.g Jinja templating for server side processing
> - `&{...}` is a preferred templating for client side processing
> - All client side context variables can be accessed using `&{ctx.env.<var>}` e.g. `&{ctx.env.USER}` returns the current OS user

Remove File(s) from Stage

Remove all files from stage `cli_stage` on path `/data`

```
snow stage remove cli_stage 'data/'  --schema=cli --database=foo
```

Native Apps

Create App

Create a Snowflake Native App `my_first_app` in current working directory,

```
snow app init my_first_app
```

Create a Snowflake Native App in directory `my_first_app`

```
snow app init --name 'my-first-app' my_first_app
```

> [!NOTE] Since the name becomes a part of the application URL its recommended to have URL safe names

Create a Snowflake Native App with Streamlit Python template[4]

```
snow app init my_first_app --template streamlit-python
```

> [!NOTE] You can also create your Snowflake Native App template and use `--template-repo` instead, to scaffold your Native App using your template.

Run App

From the application directory i.e. `cd my_first_app`

```
snow app run
```

Version App

> ![IMPORTANT] The version name should be valid SQL identifier i.e. no dots, no dashes and start with a character usually version labels use `v`.

Create Version

Create a development version named `dev`,

```
snow app version create
```

Create a development version named `v1_0`,

```
snow app version create v1_0
```

List available versions

```
snow app version list
```

Drop a Version

```
snow app version drop v1_0
```

Deploy a Version

Deploy a particular version of an application,

```
snow app run --version=v1_0
```

Deploy a particular version and patch,

```
snow app run --version=v1_0 --patch=1
```

> [!NOTE] Version `patches` are automatically incremented when creating version with same name

Open App

Open the application on a browser,

```
snow app open
```

Deploy

Synchronize the local application file changes with stage and don’t create/update the running application,

```
snow app deploy
```

Delete App

```
snow app teardown
```

If the application has version associated then drop the version,

```
snow app version drop
```

And then drop the application

```
snow app teardown
```

Drop application and its associated database objects,

```
snow app teardown --cascade
```

References

- Accelerate Development and Productivity with DevOps in Snowflake

Quickstarts

- Snowflake Developers::Quickstart
- Snowflake Developers::Getting Started With Snowflake CLI
- Build Rag Based Equipment Maintenance App Using Snowflake Cortex
- Build a Retrieval Augmented Generation (RAG) based LLM assistant using Streamlit and Snowflake Cortex

Documentation

- Snowflake CLI
- Execute Immediate Jinja Templating
- Snowpark Native App Framework
- Snowflake Cortex LLM Functions

Tutorials

- Snowflake Native App Tutorial

---

1. https://docs.snowflake.com/developer-guide/snowflake-cli-v2/connecting/specify-credentials#how-to-use-environment-variables-for-snowflake-credentials↩
2. https://docs.snowflake.com/en/user-guide/snowflake-cortex/llm-functions#choosing-a-model↩
3. https://docs.snowflake.com/en/sql-reference/sql/execute-immediate↩
4. https://github.com/snowflakedb/native-apps-templates↩